

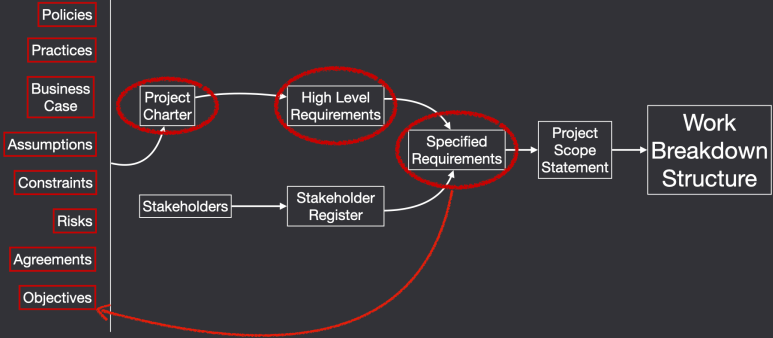
# Data and Architecture Standards

DAEN 459 / Week 10 / Fall 2025

Alexander Abuabara



# Project Charter Integration



Setting clear **standards** helps make the project easier to scale, repeat, and trust by following clear technical and organizational guidelines.

Defining a well-structured **system design and process standards** keeps the work consistent, protects data quality, and supports teamwork through every stage of the project.

# Example "Performance Analytics Pipeline"

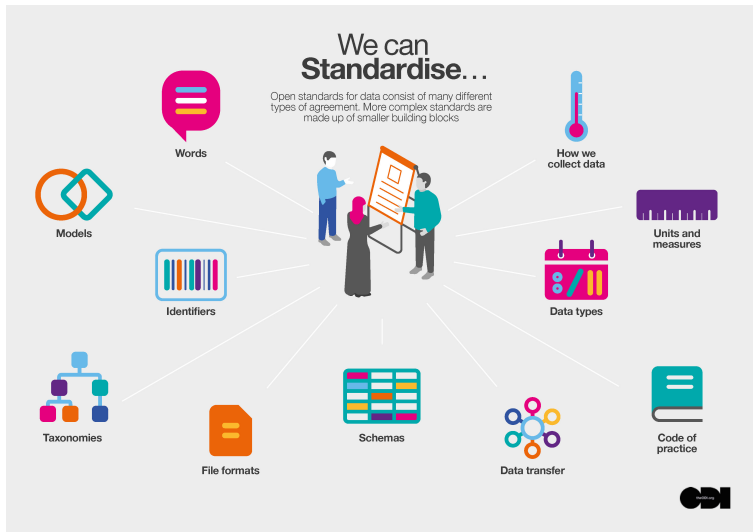
## Without Standards

- ▶ Inconsistent file formats (CSV, JSON, Excel)
- ▶ No schema validation or version control
- ▶ Code varies across team members (hard to reproduce results)

## With Clear Standards

- ▶ Defined **system design**: raw → processed data folders
- ▶ Standardized formats (Parquet), naming (`snake_case`)
- ▶ Schema validation and logging in every transformation
- ▶ Git workflow for version control and code reviews

**Result:** Reliable, scalable, and reproducible data pipeline with improved collaboration.

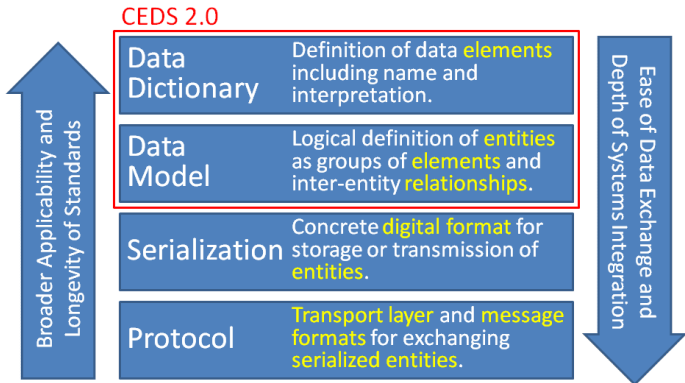


Types of open standards for data. [Source: The Open Data Institute](#)

# We can standardize

| Category          | Standard            | Description                                                                                                                         |
|-------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Shared vocabulary | Words               | Agreeing on definitions for the words we use to communicate, e.g. 'school', 'court', or 'contract'.                                 |
| Shared vocabulary | Models              | An agreed way of thinking about the types of data that we want to exchange, e.g. a concept map or an entity relationship model.     |
| Shared vocabulary | Taxonomies          | How we classify and describe things, e.g. codes and categories.                                                                     |
| Shared vocabulary | Identifiers         | The identifiers we agree to use to help us describe people, places, things and concepts in our data, e.g. a company number.         |
| Data exchange     | File formats        | The file formats we use to store and publish information, e.g. JSON, CSV, or XML.                                                   |
| Data exchange     | Schemas             | The rules for how to use a file format to exchange some data, e.g. how to use a CSV file to share planning data.                    |
| Data exchange     | Data types          | How we consistently write down different types of data, e.g. date, time and currency formats.                                       |
| Data exchange     | Data transfers      | The methods by which we exchange information or provide access to it, e.g. an API to find some data, or complete a transaction.     |
| Guidance          | How we collect data | How we consistently measure values and observe data, e.g. how to record a temperature reading, or measure a value in an experiment. |
| Guidance          | Units and measures  | The units and measurements we use to help us collect data, e.g. centigrade, lat/long, meters, etc.                                  |
| Guidance          | Codes of practice   | Best practices, recommendations, and other guidance that supports consistent data practices.                                        |

# Four-Layer Framework for Data Standards



Source: Brandt Redd, Brigham Young University.

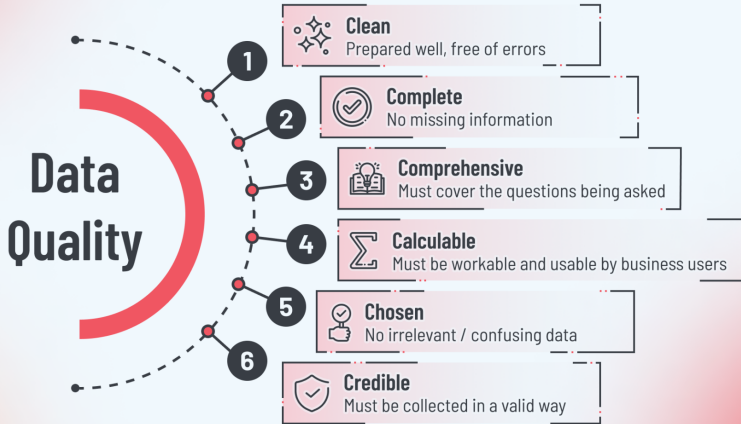
# Data Governance

- ▶ **Ownership & Stewardship:** assign clear data owners and stewards responsible for data quality, updates, and lifecycle management. Establish access permissions (read, write, admin) based on project roles.
- ▶ **Compliance:** ensure all data handling aligns with relevant regulations such as **GDPR** (privacy), **FERPA** (student records), or **HIPAA** (health data). Document data sources, consent, and usage rights.
- ▶ **Version Control:** track data and metadata changes using tools (like **Data Version Control (DVC)** or **lakeFS**). Maintain reproducibility by linking data versions to specific code commits.
- ▶ **Data Retention:** define policies for how long data is stored, archived, or deleted. Classify data by sensitivity and determine appropriate retention timelines to reduce storage costs and risk.
- ▶ **Auditability:** keep detailed logs of data access, transformations, and modifications. Enable full traceability for accountability and compliance reporting.



# Data Quality

**Means:** “fit for use” in operations and decision-making.



Source: What is Data Quality? The 6C Data- Quality Framework.

# Data Quality Dimensions

Traditionally, data quality testing is split into six dimensions:

- ▶ **Timeliness:** Is your data available when you need it?
- ▶ **Completeness:** Does it fulfill your expectations of what's comprehensive?
- ▶ **Accuracy:** How well does a piece of information reflect reality?
- ▶ **Validity (aka Conformity):** Is the information in a specific format, type, or size? Does it follow business rules/best practices?
- ▶ **Consistency:** Do data assets stored in one place match relevant data stored elsewhere?
- ▶ **Integrity (aka Uniqueness):** Can different data sets be joined correctly to reflect a larger picture? Are relationships well-defined and implemented?

# Data Documentation & Metadata

- ▶ Maintain **metadata catalogues** ([OpenMetadata](#)).
- ▶ Build **data dictionaries** defining fields, types, and logic.
- ▶ Capture **lineage tracking** ([Apache Atlas](#), [Marquez](#)).

Helps you answer questions like:

- ▶ Where did this data originate?
  - ▶ What transformations were applied?
  - ▶ Which reports or models use this dataset?
- ▶ Document assumptions, transformations, and business rules  
*(what the data means and how it should behave according to the project's real-world context).*

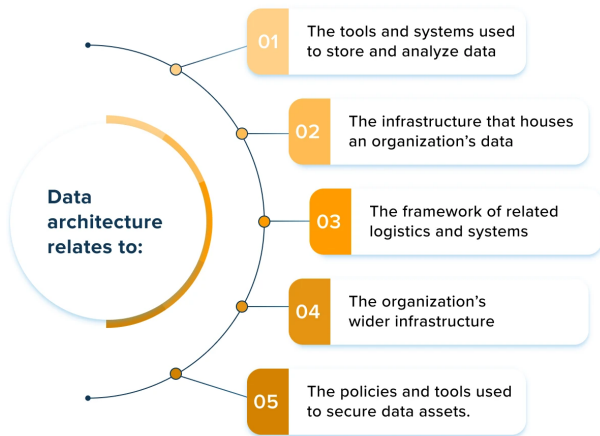
# Data Security & Privacy

- ▶ Encryption:
  - ▶ At rest: AES-256
  - ▶ In transit: TLS/SSL
- ▶ Mask or anonymize sensitive data (PII, PHI).
- ▶ Maintain access logs and audit trails.
- ▶ Implement role-based access control (RBAC).

# Data Interoperability & Formats

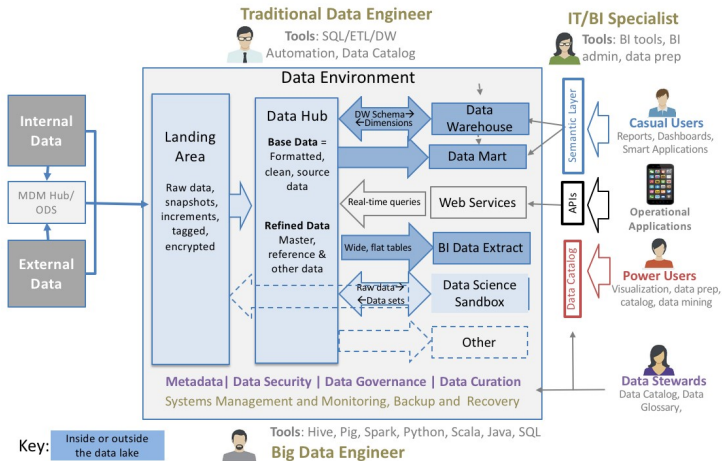
- ▶ Prefer open, efficient formats: Parquet, Avro, ORC, JSON.
- ▶ Define **schema evolution policies** for backward compatibility.  
Schema evolution policies define how your data structure (the schema: columns, types, fields) can change over time without breaking existing pipelines or applications.
- ▶ Maintain naming conventions and directory hierarchy:
  - ▶ `/raw`, `/working`, `/processed`

# Architecture Standards



Data architecture defines how an organization collects, stores, processes, and governs data across all functions. It goes beyond database design to encompass the entire data ecosystem—establishing policies, standards, and models that align data management with business goals. [Source: What Is Data Architecture: Best Practices, Strategy, & Diagram](#)

# Example



Data architecture is the blueprint for how an organization's data environment supports its goals and context. Beyond tools and products, it defines how data is captured, transformed, and delivered to users: starting from business needs and flowing back to data sources. [Source: What Is Data Architecture: Best Practices, Strategy, & Diagram](#)



# Architectural Design Principles

- ▶ **Scalability:** horizontal scaling with distributed systems.
- ▶ **Modularity:** independent ingestion, processing, storage, serving layers.
- ▶ **Fault Tolerance:** retry logic, checkpointing, idempotent writes (i.e., writing the same data or running the same operation multiple times produces the same final result — no duplicates, no corruption, no unintended side effects).
- ▶ **Observability:** logging, metrics, and tracing built-in.

# Reference Architecture

- ▶ A **reference architecture** is a high-level blueprint or template that shows how different components of a system should be organized and interact with each other.
  - ▶ It's not an exact implementation, but rather a model or guide that helps teams design consistent, scalable, and maintainable systems.
1. **Ingestion Layer:** collects raw data from multiple sources (APIs, logs, databases).
  2. **Processing Layer:** cleans, transforms, and enriches the data.
  3. **Storage Layer:** stores data in organized zones  
For instance: Raw → Curated → Analytics
  4. **Serving Layer:** delivers processed data to users or systems (dashboards, ML models, APIs, BI tools).
  5. **Governance Layer:** handles quality, security, lineage, and compliance across all stages.

# Coding & Development Standards

- ▶ Follow [Python PEP8](#), SQL style guides, and naming conventions.
- ▶ Version control with Git: feature/develop/main branches.
- ▶ Code reviews through Pull Requests.
- ▶ Implement CI/CD for data workflows (GitHub Actions, Jenkins, Dagster).

# Pipeline Design Patterns

- ▶ **ETL vs. ELT:** select the right pattern based on project needs.  
*ETL (Extract–Transform–Load)* performs transformations before loading into storage — ideal for on-prem systems.  
*ELT (Extract–Load–Transform)* loads raw data first and transforms it within the data warehouse — better for cloud-scale analytics.
- ▶ **Idempotency:** Design every pipeline step so rerunning it produces the same final result. this prevents duplication, ensures reliability, and supports recovery from job failures.
- ▶ **Partitioning and Parallelization:** split large datasets by logical keys (e.g., date, region, or customer ID) to process data in parallel, improving speed and scalability.
- ▶ **Job Orchestration:** use workflow managers to schedule, monitor, and manage dependencies between tasks for consistent, automated data flow.

# Performance Optimization

- ▶ **Define Performance KPIs:** establish clear metrics such as query latency, data throughput, and cost efficiency to evaluate system performance and identify bottlenecks.
- ▶ **Caching and Indexing:** implement caching for frequently accessed data and create indexes on high-usage columns to reduce read times and improve query speed.
- ▶ **Query Optimization:** refactor queries to minimize full table scans, leverage partition pruning, and push computations closer to data sources for efficiency.
- ▶ **Resource Scaling:** enable automatic scaling of compute resources to handle variable workloads, ensuring performance during peak demand without overspending.
- ▶ **Monitoring and Tuning:** continuously monitor job performance and storage utilization using appropriate tools, and adjust configurations based on observed metrics.

# Monitoring, Logging, & Alerting

- ▶ Maintain structured logs (INFO, WARN, ERROR).
- ▶ Define alert thresholds for performance and data quality issues.
- ▶ Include automated incident response procedures.

# Documentation & Testing

- ▶ Architecture diagrams with draw.io or Lucidchart.
- ▶ Maintain README and internal wiki per pipeline.
- ▶ **Testing Framework:**
  - ▶ Unit tests for transforms
  - ▶ Integration tests for component flows
  - ▶ Regression tests for drift detection

# Key Principles for the Project Charter

- ▶ **Consistency:** standards create reproducible and auditable outcomes.
- ▶ **Quality:** strong data governance ensures trust and usability.
- ▶ **Maintainability:** documentation and modularity prevent technical debt.
- ▶ **Scalability:** architecture designed for growth and efficiency.



